ELI MKXI — Full Project Breakdown and Assessment

Contents

A grounded, total-project read: what ELI is, its scale and shape, the architecture layer by layer, what's strong, where it's weak (with numbers), an honest verdict, and the highest-leverage work. Companion to `orchestration_and_agents.md` (agent/bus detail) and `agent_bus.md`.

> Method note: this is built from deep reads of the core modules (engine, executor, router, agent_bus, orchestrator, planners, memory, grounding gate, vision, identity, security, reasoning modes) plus a complete structural and code-health sweep (LOC distribution, debt markers, duplication, repo hygiene). It is grounded in a deep read of every package this session — claims are grounded in what was actually inspected.

---

1. What ELI is

A **100% local, model-agnostic personal AI** — no cloud, no telemetry, no API calls at runtime (one-time opt-in HuggingFace model downloads are the only network event). It bundles: GGUF inference (llama-cpp), a PySide6 desktop GUI, persistent SQLite+FTS5+FAISS memory, a knowledge graph, a 12-stage cognitive pipeline with a parallel multi-agent bus, a deterministic grounding/evidence layer to fight confabulation, local vision (Qwen2.5-VL hot-swap + Moondream co-resident), TTS/STT, OS control, a plugin system, a proactive daemon, and a LoRA self-training loop. It is a cognitive operating system for one machine, not a chat wrapper.

2. Scale & shape

~140,033 LOC across 364 Python files (`eli/`), plus 166 test files. *(2026-06-19.)*

Subsystem	LOC	Files	Role
`execution/`	23.1k	14	router + executor (action dispatch)

Subsystem	LOC	Files	Role
`runtime/`	22.6k	80	grounding, evidence, introspection, surfaces
`gui/`	20.3k	20	PySide6 desktop app
`kernel/`	14.5k	8	the engine (pipeline driver)
`cognition/`	14.0k	27	agent bus, orchestrator, inference, persona, modes
`tools/`	8.9k	30	image engine, news, etc.
`memory/`	7.0k	13	SQLite/FTS5 + FAISS + KG
`perception/`	6.8k	20	vision, STT, TTS, OS control
`core/`	6.6k	24	paths, settings, hardware profile
`planning/`	3.9k	24	proactive daemon, task planning
`learning/`	3.3k	12	LoRA self-training
`plugins/`	1.9k	28	runtime plugin manager
`world/`	1.5k	26	world event bus, local world bridge
`integrations/`,`utils/`,`contracts/`,`system/`,`cli/`	~5.5k	—	misc

Four files carry ~⅓ of the codebase: executor_enhanced.py (14.3k LOC), engine.py (13.4k), gui/eli_pro_audio_gui_MKI.py (11.0k), router_enhanced.py (7.1k). Next tier: labs_tab.py (5.7k), memory.py (4.5k), deterministic_grounding_gate.py (4.3k).

3. Architecture, layer by layer

- **Boot / hardware** — core/hardware_profile.py + core/startup_hardware_optimizer.py adapt n_ctx / gpu_layers / batch to whatever model + GPU are present (filename→ctx table; VRAM compute-buffer reservation). Model-agnostic.
- **Routing** — execution/router_enhanced.py: regex-first with LLM-intent fallback + an explicit priority pipeline → one of **208 manifest capabilities** (**183 routable**; 174 executor SUPPORTED_ACTIONS). Full reference with activation phrases: capabilities_and_actions.md.
- **Orchestration** — kernel/engine.py gates between the 12-stage cognition/orchestrator.py AgentOrchestrator (non-quick modes) and the parallel 14-agent cognition/agent_bus.py AgentBus (quick / fallback). ReAct tool-chaining for non-CHAT actions. Selection now flows through one typed ExecutionPlan (execution/execution_planner.py). See orchestration_and_agents.md for full detail.
- **Inference** — cognition/gguf_inference.py: model-path resolved from env / settings (no baked model); RLock-serialized; live runtime override; hot-swap with vision models.
- **Memory** — memory/memory.py is the shared foundation (FTS5 + FAISS + KG via recall_memory); two deliberate retrieval strategies sit on top (lightweight bus BusMemoryAgent vs heavyweight orchestrator OrchestratorMemoryAgent with HyDE + RAG + rerank). memory/knowledge_graph.py, memory/vector_store.py (FAISS, JSON meta).
- **Grounding / evidence (the crown jewel)** — runtime/deterministic_grounding_gate.py (4.3k), runtime/evidence_ledger.py, runtime/evidence_arbitration.py, runtime/control_contracts runtime/grounded_remediation.py (1.6k), cognition/output_governor.py. A layered, deterministic anti-confabulation system wrapped around the probabilistic model. Most local-LLM projects have nothing comparable.
- **Security** — runtime/security.py SecurityManager: **fail-closed shell gate** (ELI_ALLOWED_CMDS / ELI_FULL_CONTROL; unset ⇒ commands blocked), path allow-roots (ELI_ALLOW_ROOTS, defaults to project + home), app allowlist with a default-safe set, **SHA-256 custom-agent trust registry** (config/trusted_agents.json), prompt-injection guard, SQL identifier validation.
- **Self-model** — cognition/reasoning_modes.py: quick/fast/balanced all fastpath to quick, plus four private modes (chain_of_thought, self_consistency, tree_of_thoughts, constitutional_ai). Persona overlay, runtime/self_improvement.py, LoRA self-training (learning/). Emergent internal-state surfacing (autonomy pressure, "anomaly room") is intentional — see memory eli-emergent-voice.

4. What's genuinely strong

1. **The grounding / evidence layer.** Unusually rigorous; deterministic evidence gating around a probabilistic model is the right idea and well-developed. This is the part closest to genuinely frontier.
2. **Truly local + truly model-agnostic.** No call-home, hardware-adaptive, swappable models (verified — inference path carries no baked model identity).
3. **Real, fail-closed security.** Shell blocked by default, hash-gated custom agents, path/app allowlists. Most "AI agent" projects ship wide open.
4. **Breadth, integrated.** Vision, voice, OS control, memory, KG, plugins, self-training — all local, all wired into one pipeline.

5. Where it's weak (grounded in the sweep)

1. **2,565 `except Exception:` blocks (+7 bare `except:`).** The dominant structural problem. Errors are swallowed into "skipped"/fallback/empty everywhere — which is precisely why bugs surface only via runtime logs. Failures are invisible by design. Frontier software makes failures **loud in dev, quiet in prod**; this swallows them uniformly. Also the main reason the system is hard to reason about. (Only 12 TODO/FIXME markers exist — not because the code is clean, but because failures are absorbed rather than flagged.)

2. **God-files.** Two ~13–14k-line modules (`executor_enhanced.py`, `engine.py`) plus an 11k GUI. The executor is a giant if/elif action ladder. High regression surface, hard to hold in the head, painful to unit-test.
3. **Duplication & overlap.** Two separate image-engine implementations (`eli/tools/image_engine.py` 1.75k LOC *and* the `eli/tools/image_engine/image_engine/` package). `runtime/` (66 files) has many near-duplicate `personal_memory_*` / `*_surface` / `*_response` modules doing overlapping grounding work. Several plan representations coexisted (partly consolidated). Fingerprint of fast solo iteration — new code added beside the old, not folded in.
4. **Repo hygiene.** Committed junk at root: empty `...` and `[package-index-options]` files, one-off `patch_gpu_dynamic.py` / `patch_s11_bugs.py`, three `verify_eli_claims*.sh` versions, diag outputs, `.coverage`, and `experimental/*.zip` binaries. Makes the repo look less serious than the code is.
5. **Tests are GREEN (measured 2026-06-19).** 162 test files; `pytest tests/` = **6,880 passed / 42 skipped / 2 xfailed / 0 failed** (6,924 collected, ~7m25s on the `.venv`/GPU). The `tests/claims/` contract layer makes it a real safety net.
6. **13 monkeypatch/`globals()` hacks** — mostly load-bearing (e.g. the CPU-clip vision fix), but they're fragile seams worth tracking.

6. Honest verdict on "frontier, ground-breaking"

In ambition and in specific subsystems — yes. The grounding layer, the fully local model-agnostic design, and the integrated multi-modal local agent are genuinely ahead of most open local-assistant projects. The ideas are frontier-grade.

In engineering discipline — not yet. The 2,565 swallowed exceptions, the god-files, the duplication, and the clutter separate "an extraordinarily ambitious solo project" from "software others can build on." None of that is a vision problem — it is consolidation and observability. The gap to "ground-breaking" is **subtraction and discipline, not more features.**

7. Highest-leverage work (the next month)

Ranked by effect-per-effort:

1. **Tame error-swallowing.** Replace blanket `except Exception: pass`/fallback with scoped exception types + a single structured error log (a ring buffer / table) you can actually watch. Keep the graceful degradation, but record every swallow. This alone makes the whole system debuggable and is the precondition for trusting any other change.
2. **Split the two god-files** along their natural seams (executor: action groups into per-domain modules behind the dispatch table; engine: pipeline stages into stage modules). Reduces regression surface and makes the pipeline readable.
3. **Delete duplication + clutter; get tests green.** Collapse the two image engines, remove root junk/one-off scripts, fix or quarantine the ~24 failing tests. Cheap, high signal-to-noise improvement.
4. **Consolidate the `runtime/` surfaces.** The many `personal_memory_*` / `*_surface` / `*_response` modules want to be a handful of well-named ones.

Do 1–3 and the engineering would match the ideas — which is the only thing standing between ELI and the label "frontier, ground-breaking software people can rely on."